BUILDING AND USING THE JAVA SDK FOR COPILOT USING AI AGENTS

GitHub Copilot is a programmable AI platform — let me show you how!

☕ Java 17+ | 🤖 AI-Assisted Dev | 🔧 Open Source | ⚡ JSON-RPC

JavaOne | 2026

github.com/github/copilot-sdk-java

WHAT WE'LL COVER

🏗️ The SDK

- Copilot as a programmable platform
- Architecture & core concepts
- Sessions, events, tools, MCP
- Live code walk-through

🤖 Built with AI Agents

- Agentic upstream sync
- Documentation assessment
- Test coverage checks
- PR automation from start to finish

> *Whether you want to **extend Copilot** or rethink how you **ship software with AI**, this talk has something for you.*

GITHUB COPILOT IS MORE THAN AN EDITOR PLUGIN

📝 CODE COMPLETION

What most developers use today: inline suggestions, chat, code review.

⚙️ COPILOT CLI

A standalone process you can **spawn and control programmatically** — the foundation of the SDK.

🤖 COPILOT CODING AGENT

Autonomous agent that can open PRs, fix issues, run tests — orchestrated through GitHub.

The Copilot CLI exposes a **JSON-RPC API over stdio**.

The SDK wraps that API with idiomatic Java — sessions, events, tools.

COPILOT SDK FOR JAVA

- Unofficial, community-driven port of the official .NET SDK
- Tracks upstream .NET reference implementation
- Available on **Maven Central**
- Java 17+ — uses modern language features
- Minimal dependencies (Jackson only)
- MIT licensed

```
<dependency>
  <groupId>com.github</groupId>
  <artifactId>copilot-sdk-java</artifactId>
  <version>0.1.32-java.0</version>
</dependency>
```

CORE CAPABILITIES

- Conversational AI sessions
- Streaming responses
- Custom tool registration
- MCP server integration
- Session persistence & resume
- Multiple concurrent sessions
- Model selection per session
- Custom agent selection

ARCHITECTURE

Your Java Application

CopilotClient · CopilotSession · ToolDefinition

⬇ JSON-RPC over stdio ⬆

GitHub Copilot CLI

Local process — spawned automatically

⬇ HTTPS ⬆

GitHub Copilot API

LLM inference · Tool orchestration · MCP routing

The CLI handles auth, protocol, model routing. Your app just **sends messages and handles events**.

HELLO, COPILOT! DEMO

```java
// Works with JBang — no project setup needed!
// //DEPS com.github:copilot-sdk-java:0.1.32-java.0

import com.github.copilot.sdk.*;
import com.github.copilot.sdk.events.*;
import com.github.copilot.sdk.json.*;

try (var client = new CopilotClient()) {
    client.start().get();                                    // spawn Copilot CLI

    var session = client.createSession(
        new SessionConfig()
            .setOnPermissionRequest(PermissionHandler.APPROVE_ALL)
            .setModel("claude-sonnet-4.5")
    ).get();

    var response = session.sendAndWait(
        new MessageOptions().setPrompt("What is 2 + 2?")
    ).get();

    System.out.println(response.getData().content());          // 4
```

Three objects: `CopilotClient` · `CopilotSession` · `MessageOptions`

STREAMING RESPONSES & EVENTS

```java
var done = new CompletableFuture<Void>();

// Print each word as it arrives
session.on(AssistantMessageDeltaEvent.class, delta -> {
    System.out.print(delta.getData().deltaContent());     // streaming chunk
});

// Know exactly when the model is done
session.on(SessionIdleEvent.class, idle -> {
    System.out.println();
    done.complete(null);
});

// Track token consumption
session.on(SessionUsageInfoEvent.class, usage -> {
    var d = usage.getData();
    System.out.printf("Tokens: %d / %d%n",
        (int) d.currentTokens(), (int) d.tokenLimit());
});

session.send(new MessageOptions().setPrompt("Tell me a haiku about Java")).get();
```

30+ event types: assistant messages, tool calls, errors, model changes, plan mode…

CUSTOM TOOLS DEMO

```
// Give Copilot the ability to call your code
var weatherTool = ToolDefinition.create(
    "get_weather",
    "Return current weather for a city",
    Map.of(
        "type", "object",
        "properties", Map.of(
            "city", Map.of("type", "string", "description", "City name")
        ),
        "required", List.of("city")
    ),
    invocation -> {
        String city = (String) invocation.getArguments().get("city");
        String weather = MyWeatherService.fetch(city);   // your logic
        return CompletableFuture.completedFuture(
            new ToolResult().setContent(weather)
        );
    }
);

var session = client.createSession(
```

Copilot decides *when* to call the tool; you define *what* it does.

MCP SERVERS — PLUG IN EXTERNAL TOOLS

Mount a **Model Context Protocol** server and Copilot gains tools automatically.

```
// Give Copilot access to your filesystem
Map<String, Object> fsServer = Map.of(
    "type",    "local",
    "command", "npx",
    "args",    List.of(
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/tmp"
    ),
    "tools",   List.of("*")
);

var session = client.createSession(
    new SessionConfig()
        .setOnPermissionRequest(
            PermissionHandler.APPROVE_ALL)
        .setMcpServers(Map.of("fs", fsServer))
).get();
```

POPULAR MCP SERVERS

- **filesystem** — read/write files
- **github** — issues, PRs, repos
- **sqlite** — query databases
- **puppeteer** — browser automation
- **postgres** — PostgreSQL queries
- … 200+ in the community registry

REMOTE MCP (HTTP/SSE)

```
Map.of(
  "type", "http",
  "url",  "https://api.example.com/mcp",
  "headers", Map.of(
    "Authorization", "Bearer " + token),
  "tools", List.of("*")
)
```

MORE SESSION FEATURES

🔁 RESUME SESSIONS

```
// Save
String id = session.getSessionId();

// Later…
var s2 = client.resumeSession(
  new ResumeSessionConfig()
    .setSessionId(id)
    .setOnPermissionRequest(...)
).get();
```

🔀 MULTI-SESSION

```
// Run sessions concurrently
var coder = client.createSession(
    new SessionConfig()
      .setModel("gpt-5")).get();

var reviewer = client.createSession(
    new SessionConfig()
      .setModel("claude-sonnet-4.5")
).get();
```

🗜️ COMPACT

```
// Summarise history to
// stay within token limit
session.compact().get();
```

🔄 SWITCH MODEL

```
// Hot-swap model mid-chat
session.setModel("o3").get();
```

SECURITY: PERMISSION HOOKS

```java
// Every createSession / resumeSession MUST provide a permission handler
var session = client.createSession(
    new SessionConfig()
        .setOnPermissionRequest((request, invocation) -> {
            // request.getPermissionKind() — what is being asked
            // invocation.getToolName()    — which tool wants permission
            // invocation.getArguments()  — what arguments it will use

            if (request.getPermissionKind().equals("write_file")) {
                boolean ok = promptUser("Allow writing to " + invocation.getArguments().get("path") + "?");
                return CompletableFuture.completedFuture(
                    new PermissionRequestResult().setKind(ok ? "allow" : "deny")
                );
            }

            // Approve everything else (convenient for development)
            return CompletableFuture.completedFuture(
                new PermissionRequestResult().setKind("allow")
            );
        })
).get();
```

5 hook types: `onPermissionRequest` · `onBeforeToolExecution` · `onAfterToolExecution` · `onBeforeUserInput` · `onUserInput`

REAL-WORLD USAGE: JMETER COPILOT PLUGIN

The **JMeter Copilot Plugin** (github.com/brunoborges/jmeter-copilot-plugin) adds an AI-powered load-test assistant to Apache JMeter.

- Runs as a JMeter extension (JAR on classpath)
- Creates a `CopilotSession` on plugin startup
- Users describe test scenarios in plain English
- Copilot generates JMeter test plans as JSON / XML
- Plugin parses the response and injects it into the JMeter UI

💡 WHAT YOU CAN BUILD

- JBang CLI utilities
- Maven / Gradle plugins
- Spring Boot AI services
- IDE extensions
- CI/CD pipeline assistants
- Documentation generators
- Code review bots
- Test generation tools

🤖 BUILDING THE SDK WITH AI AGENTS

How we used **GitHub Copilot's own agentic capabilities**
to build and maintain the SDK

Upstream Sync | Documentation | Test Coverage | PR Automation

AGENTIC UPSTREAM SYNC AI

The SDK tracks the official **.NET reference implementation**.

- 1 Weekly GitHub Actions cron job checks for new upstream commits
- 2 If changes found, **creates an issue** labeled `upstream-sync`
- 3 **Assigns it to the Copilot coding agent**
- 4 Agent reads the .NET diff, ports changes to Java idioms
- 5 Opens a PR with tests, docs, and CHANGELOG entry
- 6 Closes the old issue automatically

THE .LASTMERGE FILE

Tracks the last upstream commit SHA.

```
a3f91cb
```

Each release records the upstream sync point in `CHANGELOG.md`.

PROMPT-AS-CODE

The merge workflow is a `.prompt.md` file — 11 explicit steps, referenced by VS Code Copilot Chat as a slash command.

DOCUMENTATION COVERAGE ASSESSMENT AI

A reusable prompt asks the agent to **audit documentation** against the public API.

WHAT THE AGENT CHECKS

- Every public method has Javadoc
- All event types are listed in docs
- New features have code examples
- Getting-started guide is up to date
- Cookbook recipes compile against current API

OUTCOME

- Gaps filed as GitHub issues
- Agent opens PRs with new docs
- `DocumentationSamplesTest` guards against regressions:
  - Scans README, jbang-example, site docs
  - Rejects removed API usage
  - Enforces required handler patterns

PR AUTOMATION — END TO END AI

Every pull request in this repo follows an agentic workflow:

1. BRANCH & CODE

Agent creates a feature branch, implements the change, runs `mvn spotless:apply`.

2. TEST & REVIEW

Runs `mvn verify`, triggers **code_review** and **codeql_checker** tools, addresses feedback.

3. SHIP IT

commit-as-pull-request skill: creates branch, pushes, opens PR, squash-merges, syncs `main`.

> *The agent uses the SDK it is maintaining to orchestrate its own merge workflows — meta AI engineering.*

SKILLS = REUSABLE PROMPTS

Custom skills defined as `.prompt.md` files are registered in `.github/copilot-instructions.md`.

AVAILABLE SKILLS

- `/agentic-merge-upstream` — port .NET changes to Java
- `/documentation-coverage` — audit docs vs public API
- `/commit-as-pull-request` — automate the full PR workflow

Skills can be invoked from:

- **VS Code Copilot Chat** — `/skill-name`
- **GitHub Copilot Coding Agent** — assigned via issue labels
- **Workflow dispatch** — triggered from GitHub Actions UI

💡 KEY INSIGHT

Treating prompts as **versioned, testable code** (not chat history) is what makes agentic workflows *repeatable and maintainable*.

LESSONS LEARNED

✅ What worked well

- Prompts as versioned code in the repo
- Explicit numbered steps in prompts (less ambiguity)
- Guard tests that prevent documentation drift
- Having the SDK *eat its own dog food* in agents
- Minimal dependency footprint — easy to review agent PRs
- Sealed event hierarchy — great for exhaustive matching

⚠️ Watch out for

- Agent PRs still need human review for nuanced .NET → Java translation
- Context window limits on large upstream diffs
- Agent can be overly eager to add dependencies
- Documentation tests can be brittle if API changes fast
- Always verify security-sensitive permission handler examples

🧪 HANDS-ON LAB

Build a Copilot-powered CLI in 15 minutes with **JBang** — no project setup needed.

OPTION A — JBANG CLI

A single `.java` file that asks Copilot questions from the terminal. Add custom tools, streaming, and session persistence.

```
jbang init MyCopilot.java
# add //DEPS header
# run with: jbang MyCopilot.java
```

OPTION B — MAVEN PLUGIN

A Maven plugin that adds an AI-assisted `copilot:review` goal to any project.

```
mvn archetype:generate \
  -DarchetypeArtifactId=\
    maven-archetype-plugin
# add SDK dependency
# run: mvn copilot:review
```

Lab instructions → docs/javaone/lab.md in the repository

RESOURCES

SDK

- github.com/github/copilot-sdk-java
- Documentation site
- Maven Central
- Javadoc

Related

- Official .NET SDK (github/copilot-sdk)
- Copilot CLI Installation
- Model Context Protocol
- JBang

Try it right now

```
# Run the example directly from GitHub
jbang https://github.com/github/copilot-sdk-java/blob/latest/jbang-example.java
```

Lab files

```
docs/javaone/
  slides.html   ← you are here
  lab.md        ← hands-on instructions
```

THANK YOU! 🙏

GitHub Copilot is a **programmable AI platform**.

The Java SDK lets you build with it natively in the JVM ecosystem.

And the whole thing was built — and is maintained — with AI agents.

⭐ Star the repo | 📖 Read the docs | 🧪 Try the lab | 🤝 Contribute

github.com/github/copilot-sdk-java

⚠️ Unofficial, community-driven SDK. Not supported or endorsed by GitHub.